

Assignment 2: Text Preprocessing Pipeline - NLTK vs spaCy

(strict action and policy regarding plagiarism and ai generated code/content – lead to no refund or certificate generation) (use ai tools for help and write your own codes)

Title

Building and Benchmarking Text Preprocessing Pipelines using NLTK and spaCy

Problem Statement

Develop comprehensive text preprocessing pipelines using both NLTK and spaCy libraries, then compare their performance, accuracy, and use cases. You will handle real-world noisy text from social media, implement standard NLP preprocessing steps, and create a reusable preprocessing library for future projects.

Dataset

Twitter Sentiment Analysis Dataset

- **Source:** [Sentiment140](#) or [Twitter US Airline Sentiment](#)
- **Description:** Tweets with sentiment labels, contains hashtags, mentions, emojis, URLs
- **Size:** 1.6 million tweets (use subset of 10,000 for this assignment)

Alternative: Reddit Comments Dataset

- **Source:** [Reddit Comments Dataset](#)

Detailed Instructions

Part 1: Environment Setup and Data Loading (10%)

1. Install required libraries:

```
pip install nltk spacy pandas matplotlib seaborn
```

```
python -m spacy download en_core_web_sm
```

2. Download NLTK data:

```
import nltk
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger')
```

3. Load Twitter dataset and explore:

- Check for missing values
- Display sample tweets (5 from each sentiment)
- Analyze tweet length distribution
- Count hashtags, mentions, URLs

Part 2: Text Cleaning (15%)

Implement cleaning functions that:

1. **Lowercase conversion**

2. **Remove elements:**

- URLs (http://..., https://...)
- Mentions (@username)
- Hashtags (option to keep or remove)
- Numbers (or keep based on context)
- Special characters and punctuation
- Extra whitespace

3. **Handle emojis:**

- Option A: Remove completely
- Option B: Convert to text description using emoji library
- Compare both approaches

4. **Expand contractions:** "don't" → "do not", "it's" → "it is"

5. **Handle repeated characters:** "sooooo happy" → "so happy"

Create a `clean_text()` function that applies all steps.

Part 3: NLTK Preprocessing Pipeline (25%)

Task 3.1: Tokenization

1. Word tokenization using `word_tokenize()`
2. Sentence tokenization using `sent_tokenize()`
3. Handle edge cases (URLs, mentions, hashtags)
4. Compare with simple `split()` method

Task 3.2: Stopword Removal

1. Use NLTK's English stopwords list
2. Create custom stopwords list (add domain-specific words)
3. Filter tokens while preserving sentiment words (don't remove "not", "no", "never")

Task 3.3: Stemming vs Lemmatization

1. Apply Porter Stemmer
2. Apply Snowball Stemmer
3. Apply WordNet Lemmatizer (with POS tagging)
4. Compare results:
 - "running", "runs", "ran" → ?
 - "better", "good", "best" → ?

Task 3.4: Part-of-Speech Tagging

1. Apply POS tagging to tokens
2. Extract only nouns and adjectives
3. Analyze POS distribution

Task 3.5: Complete NLTK Pipeline

```
def nltk_preprocess(text):  
    # Clean  
    # Tokenize  
    # Remove stopwords  
    # Lemmatize with POS  
    # Return processed text  
    pass
```

Part 4: spaCy Preprocessing Pipeline (25%)

Task 4.1: Basic Processing

1. Load spaCy model: `nlp = spacy.load('en_core_web_sm')`
2. Process text: `doc = nlp(text)`
3. Access tokens, lemmas, POS tags in one pass

Task 4.2: Advanced Features

1. **Lemmatization:** Access via `token.lemma_`
2. **POS Tagging:** Access via `token.pos_`
3. **Dependency Parsing:** Visualize syntactic dependencies

4. Named Entity Recognition:

- Extract entities (PERSON, ORG, GPE, etc.)
- Count entity types
- Create entity frequency chart

Task 4.3: Custom Pipeline Components

1. Add custom stopwords
2. Create custom entity ruler
3. Implement custom filtering logic

Task 4.4: Complete spaCy Pipeline

```
def spacy_preprocess(text, nlp):  
    doc = nlp(text)  
    # Filter stopwords, punctuation  
    # Lemmatize  
    # Extract based on POS  
    # Return processed text  
    pass
```

Part 5: Comparative Benchmarking (25%)

Task 5.1: Performance Comparison

1. Processing Speed:

- Time both pipelines on 1000, 5000, 10000 tweets
- Plot processing time vs dataset size
- Calculate tweets per second

2. Memory Usage:

- Monitor RAM consumption
- Compare memory efficiency

3. Accuracy Comparison:

- Compare lemmatization quality (manual inspection of 100 samples)
- Compare POS tagging accuracy
- Check NER accuracy (if applicable)

Task 5.2: Output Quality Analysis

1. Create comparison table:

Metric	NLTK	spaCy
Processing time (10k tweets)	X sec	Y sec
Average tokens per tweet	A	B
Unique tokens after preprocessing	C	D
Lemmatization quality	Rating	Rating

2. Visualize differences:
 - Bar chart comparing processing times
 - Venn diagram of vocabulary overlap
 - Token count distributions

Task 5.3: Use Case Recommendations

Create a decision matrix:

- When to use NLTK?
- When to use spaCy?
- Hybrid approach scenarios

Expected Outputs

1. Code Deliverables

- preprocessing_library.py with reusable functions
- nltk_pipeline.py - Complete NLTK implementation
- spacy_pipeline.py - Complete spaCy implementation
- benchmark.py - Performance comparison script
- Jupyter notebook with experiments and visualizations

2. Processed Data

- CSV files with original and preprocessed tweets (both pipelines)
- JSON file with extracted entities

3. Visualizations (minimum 10)

- Tweet length distribution (before/after preprocessing)
- Processing time comparison (bar chart)

- Token count distribution
- POS tag distribution (pie chart)
- Named entity frequency (horizontal bar chart)
- Stopword removal impact
- Stemming vs Lemmatization comparison
- Word cloud (before and after preprocessing)
- Memory usage over time
- Dependency parse tree examples (spaCy)

4. **Benchmarking Report** (3-4 pages)

- Methodology
- Performance metrics table
- Quality comparison with examples
- Pros and cons of each library
- Recommendations for different use cases
- Code snippets showing key differences

Findings/Observations to Document

1. How does preprocessing affect vocabulary size?
2. Which library is faster for small vs large datasets?
3. What are the key differences in lemmatization quality?
4. How accurate is spaCy's NER on social media text?
5. What preprocessing steps have the most impact on text length?
6. Are there cases where stemming is preferable to lemmatization?
7. How well do standard stopwords lists work for Twitter data?
8. What challenges are unique to social media text preprocessing?

Evaluation Rubric (100 points)

Criteria	Points	Description
NLTK Pipeline	25	Complete, correct implementation of all steps
spaCy Pipeline	25	Complete, correct implementation with advanced features
Benchmarking	20	Thorough performance and quality comparison
Code Quality	15	Reusable library, clean code, proper documentation
Visualizations	10	Clear, informative plots with proper interpretation
Report	5	Well-structured analysis with actionable insights

Bonus Points (10)

- Implement custom spaCy pipeline component (+3)
- Add support for multiple languages (+3)
- Create performance profiling with line_profiler (+2)
- Implement parallel processing for batch preprocessing (+2)
